STYLUS — Guía Técnica de la Plataforma

Retail con compras inteligentes asistidas por IA en tiempo real · UTEC
Taller de Arquitectura Big Data

24 de octubre de 2026

Table of Contents

1. Qué es STYLUS

STYLUS es una plataforma de Big Data en **Google Cloud** que procesa el dataset público y real de Kaggle **H&M Personalized Fashion Recommendations** (31,788,324 transacciones, 1,371,980 clientes, 105,542 artículos) y lo convierte en: un lakehouse por capas (Bronze/Silver/Gold), un motor de recomendaciones, un asistente de compras con Vertex AI Gemini y una tienda web con pago simulado. Repositorio: github.com/estuardoo/STYLUS (privado).

1.1 Equipo y responsabilidades

Integrante	Perfil	A cargo de
Estuardo Oliver	ML / Analytics	Asistente Gemini, recomendaciones, documentos de entrega, matriz de auditoría
Winston Quispe	Cloud Computing	Terraform, GCP, Pub/Sub, Dataflow, CI/CD, costos, orquestador ops.py
Nicole Arenas	Datos + API + Modelado	Bronze/Silver/Gold, modelo estrella, FastAPI, Cloud SQL
Gianmarco Mejia	Dashboard / BI	KPIs, Looker Studio, calculadora y tablero de costos

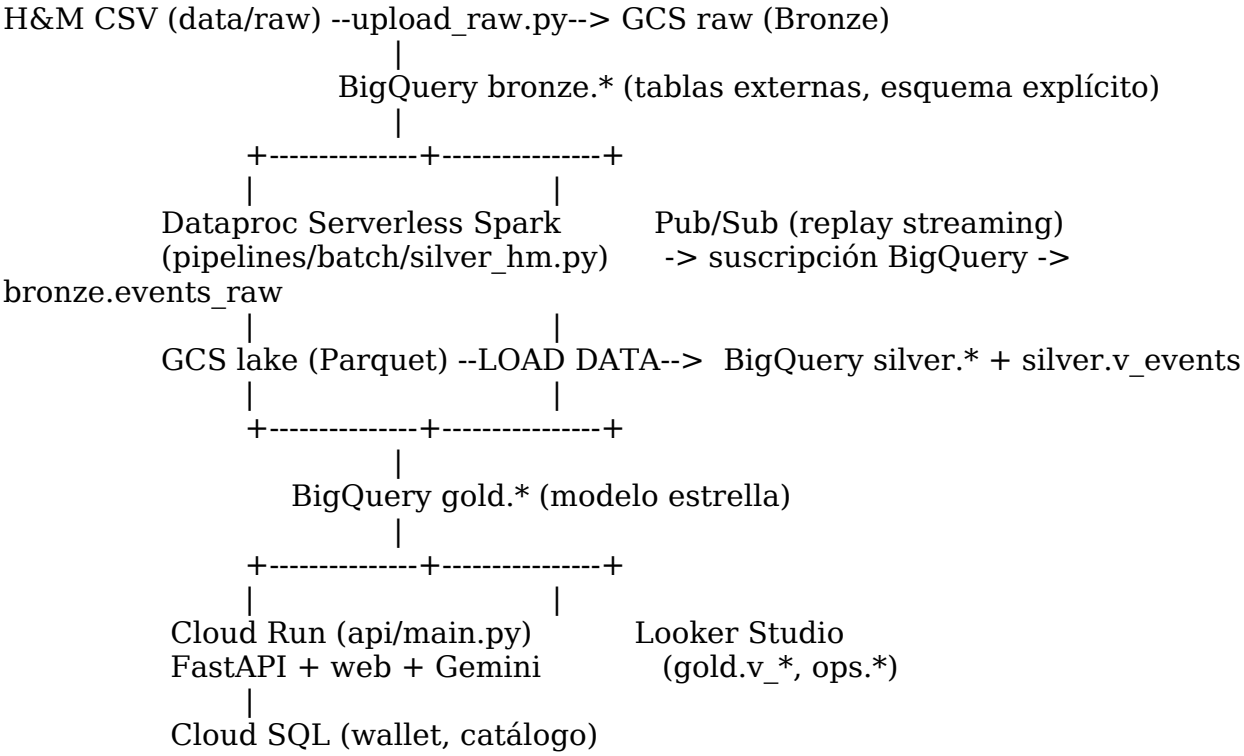
2. Fuente de datos

Todo el proyecto corre sobre datos **públicos y reales**: el dataset de Kaggle H&M Personalized Fashion Recommendations. No se usan datos propios ni sintéticos para las entidades base.

Archivo original	Filas	Contenido
articles.csv	105,542	Catálogo de productos
customers.csv	1,371,980	Clientes
transactions_train.csv	31,788,324	Compras históricas
images/	105,100	Fotos de producto (no se usan; excluidas de la subida por defecto)

Los eventos de streaming de 2026 son **re-emisiones** de esas mismas transacciones reales (source_t_dat conserva la fecha original; is_synthetic=true), nunca datos inventados.

3. Arquitectura



Decisión por capa y alternativas descartadas: ver README.md y el documento de Entrega 1 §5.

4. Fuente única de configuración: config/control_plataforma.json

Reemplaza lo que antes era un Excel. Contiene, en una sola estructura validada:

Sección	Contenido	Filas
parametros	project_id, región, Cloud SQL, Cloud Run,	64 (38 editables)

	Dataflow, modelo Gemini, costos	
apis	Servicios de GCP a activar	21
cuentas_servicio	Cuentas de servicio y su propósito	4
permisos	Roles IAM por recurso	30
usuarios	Equipo (roles GCP) + usuarios de la tienda (saldo inicial)	12
datasets / tablas / columnas	Esquema de BigQuery (contrato con el SQL real)	4 / 30 / 240
secretos	Nombres de secretos (el valor lo crea Terraform)	2

python ops.py config lee este JSON, lo valida (tipos, roles, referencias cruzadas) y genera de forma determinística: - infra/terraform/plataforma.auto.tfvars.json (variables de Terraform) - config/generated/plataforma.json (configuración para ops.py y la API) - api/seed.json (usuarios iniciales de la tienda)

python ops.py verificar compara el INFORMATION_SCHEMA real de BigQuery contra este JSON y marca OK / FALTA / DIFERENTE tabla por tabla.

4.1 Panel de control web

python ops.py panel levanta http://127.0.0.1:8765 (solo en tu máquina). Permite: - Editar los 38 parámetros marcados como editables, con su descripción. - Editar como tabla (agregar/quitar fila) usuarios, permisos, cuentas de servicio y APIs. - Ver de solo lectura datasets/tablas/columnas/secretos (son contrato con el SQL y Terraform). - Ejecutar cualquier comando de ops.py con un botón y ver el registro en vivo (Server-Sent Events). - Los comandos que modifican o borran infraestructura real (apply, destroy) piden confirmación explícita.

5. Orquestador ops.py

Un comando por paso del pipeline, multiplataforma (Windows/Linux/Mac/Cloud Shell), con un registro de tiempos en formato de tabla al final de cada ejecución.

Comando	Qué hace
config [--check]	Valida el JSON y regenera los archivos derivados
estado	Muestra los nombres y valores efectivos (buckets, datasets, URL de la API, etc.)

local	Tienda en http://localhost:8080 con el catálogo H&M local (SQLite), sin GCP
bootstrap	Activa APIs, red (Private Google Access), bucket de estado de Terraform, terraform init
plan / apply / destroy [--yes]	Ciclo de vida de la infraestructura
subir-datos [--con-imagenes]	Sube data/raw a GCS (rsync idempotente con manifiesto SHA-256)
bronze / silver / gold	Las tres capas del lakehouse
api	Imagen (Cloud Build) + Cloud Run + carga del catálogo a Cloud SQL
replay [--limite N]	Re-emite transacciones históricas como eventos de streaming 2026
dataflow-iniciar / dataflow-detener	Encender/apagar el pipeline de streaming (cobra mientras corre)
verificar	BigQuery real vs. JSON de control + prueba de /health y /ready
github	Imprime el valor de GCP_WIF_PROVIDER para configurarlo en GitHub Actions
costo-real	Vista de costo real por pedido (requiere billing export activado)
panel	Panel web local descrito en §4.1
todo [--yes]	Encadena bootstrap → apply → bronze → silver → gold → api → verificar

6. Ingesta y almacenamiento

- **Batch:** ingestion/batch/upload_raw.py calcula SHA-256 y número de filas de cada CSV, genera un manifiesto (data/manifests/hm_raw_manifest.json) y sube con gcloud storage rsync (idempotente: solo copia lo que cambió). Por defecto excluye las 105,100 imágenes (--con-imagenes las incluye).
- **Streaming:** ingestion/streaming/replay_publisher.py re-emite transacciones históricas como eventos Pub/Sub con fecha 2026, esperando la confirmación de cada publicación antes de continuar.
- **Bronze:** tablas externas sobre los CSV en GCS, con **esquema explícito** (no autodetectado, para no perder ceros a la izquierda en article_id) — sql/bronze/01_bronze_hm.sql.
- **Silver:** pipelines/batch/silver_hm.py (PySpark, Dataproc Serverless) limpia tipos y escribe Parquet particionado en GCS; sql/silver/01_load_silver.sql lo carga a BigQuery con LOAD DATA OVERWRITE. Los eventos de streaming llegan por una

ruta económica: suscripción Pub/Sub → BigQuery directo (bronze.events_raw), expuestos como la vista silver.v_events (deduplicada por event_id).

- **Gold:** modelo estrella — ver §7.

7. Modelo de datos (Gold)

Tabla	Grano	Origen
dim_customer	1 cliente	silver.customers
dim_article	1 artículo	silver.articles
dim_date	1 día (2018–2027)	Calendario generado
dim_channel	1 canal (Tienda física / Online)	Supuesto propio (H&M no documenta los canales)
fact_sales	1 línea de venta	silver.transactions (histórico) ∪ silver.v_events (streaming)
fact_orders	1 pedido (cliente + día + fuente; H&M no trae order_id)	Agregado de fact_sales
reco_popular	1 artículo por grupo	Popularidad de los últimos 7 días (recomendador baseline)
v_catalog_serving	Vista	Catálogo que copia api/load_catalog.py a Cloud SQL

Diagrama completo con las relaciones: README.md (sección “Modelo de datos”).
Diccionario de campos: docs/diccionario_datos.md.

8. API, asistente y tienda

- api/main.py (FastAPI): catálogo, recomendaciones, asistente, carrito, checkout (ACID, idempotente, rechaza con 409 si no hay saldo), wallet (billetera append-only), recarga de saldo (con token de administrador), pedidos, métricas, imágenes.
- api/assistant.py: reglas en español que mapean a tipos de artículo H&M, con respaldo cuando no hay Vertex AI disponible; con assistant_mode = gemini usa Vertex AI Gemini 3.1 Flash-Lite (gemini-3.1-flash-lite, el modelo 2.5 se retiró de Vertex el 20-oct-2026).
- api/load_catalog.py: carga el catálogo desde BigQuery (v_catalog_serving) o, en local, desde el CSV de transacciones (popularidad de los últimos 28 días).
- Persistencia: Cloud SQL (Postgres 15) para wallet y catálogo de servicio; Secret Manager para la cadena de conexión y el token de administrador.

- Despliegue: Cloud Run (API/web) + Cloud Run Job (carga del catálogo), imagen construida con Cloud Build.

9. Costos

cost/assumptions.yaml es la única fuente de supuestos; python cost/estimate_costs.py genera el Excel con fórmulas (--refresh-prices actualiza tarifas desde la API de Cloud Billing Catalog).

Escenario
Desarrollo
Demo
Producción
Pico (Black Friday)

Terraform crea un presupuesto con alertas al 50/90/100% cuando billing_account está definido. cost/real_cost_per_order.sql calcula el costo real una vez activado el billing export a BigQuery.

10. Infraestructura (Terraform)

Archivo	Contenido
main.tf	Buckets (raw/lake), datasets, tablas base, Pub/Sub + DLQ, cuentas de servicio, IAM genérico, Artifact Registry, presupuesto
cloudsql.tf	Instancia Postgres 15 con PITR, base de datos, usuario, secretos generados (contraseña y token de administrador)
cloudrun.tf	Servicio de Cloud Run con variables de entorno, secretos montados, volumen de Cloud SQL, acceso público
cicd.tf	Workload Identity Federation para GitHub Actions (despliegue sin llaves JSON)
variables.tf / outputs.tf	Tipos de variables (alimentados por plataforma.auto.tfvars.json) y salidas (URL de la API, buckets, datasets)

Validado con OpenTofu (tofu validate, tofu fmt); ops.py detecta automáticamente terraform o tofu.

11. Seguridad y gobierno

- Nunca se versionan datos, credenciales ni *.tfstate — lo valida una regla de la CI.

- Contraseñas y tokens nunca van en config/control_plataforma.json: Terraform los genera en Secret Manager.
- roles/owner no se puede otorgar por API al equipo; se asignan roles específicos desde la sección usuarios.
- El despliegue desde GitHub Actions usa Workload Identity Federation (sin llaves de cuenta de servicio).
- Precio, stock y saldo de la tienda son simulados; ninguna compra se ejecuta sin confirmación explícita.

12. CI/CD

- .github/workflows/ci.yml: bloquea archivos prohibidos (CSV, Parquet, ZIP, .db, .tfstate, credenciales), corre python ops.py config --check, las pruebas unitarias, la calculadora de costos y terraform fmt/validate.
- .github/workflows/deploy.yml: lee config/generated/plataforma.json, construye y publica la imagen, despliega a Cloud Run — disparado por workflow_dispatch o push a api/ o config/.

13. Pruebas

python -m unittest discover -s tests — 7 pruebas: configuración válida y generados al día, detección de errores de validación, contrato de Cloud SQL contra api/schema.sql, API (incluye reseed idempotente), replay, núcleo de negocio (checkout/wallet).

14. Estado actual y próximos pasos

Ver docs/gestion/Matriz_Auditoria_Proyecto.xlsx para el detalle tarea por tarea. En resumen:

Paso
1
2
3

El bloqueador principal para avanzar P2 y P3 es ejecutar python ops.py bootstrap/apply/bronze/silver/gold/api contra el proyecto real de GCP (tranquil-leaf-512618-b9): el diseño, el código y las pruebas locales ya están listos para esos pasos.